

# Lecture 11: Floating-Point Arithmetic and Round-Off Errors

Math 6610

C. Mou

Department of Mathematics and Statistics  
Utah State University

Fall 2026

# Today's Goals

- Read the stored exponent and the 64-bit layout of a double
- Recognize the special bit patterns:  $\pm 0$ ,  $\pm \infty$ , NaN, subnormal numbers
- State the rule for one operation:  $x \circledast y = (x * y)(1 + \epsilon)$ ,  $|\epsilon| \leq \frac{1}{2} \epsilon_{\text{mach}}$
- Measure the loss of digits in one subtraction

# The Stored Exponent

**Definition (*Exponent field*).** Double precision stores the exponent  $p$  in 11 bits  $e_{10}, e_9, \dots, e_0 \in \{0, 1\}$ . Read as an integer,

$$E = e_{10}2^{10} + e_92^9 + \dots + e_12^1 + e_02^0, \quad 0 \leq E \leq 2^{11} - 1 = 2047.$$

The exponent is  $E$  shifted by a fixed amount:

$$p = E - 1023, \quad \text{so} \quad -1023 \leq p \leq 1024.$$

**Remark (*Reserved values*).**  $E = 0$  ( $p = -1023$ ) and  $E = 2047$  ( $p = 1024$ ) are reserved for special numbers (next slides). Normal numbers use  $1 \leq E \leq 2046$ , i.e.  $-1022 \leq p \leq 1023$ , as in Lecture 10.

# Memory Layout of a Double

64 bits: 1 sign bit, 11 exponent bits, 52 mantissa bits (mantissa = the bits after the binary point):

$$\boxed{s \mid e_{10} e_9 \cdots e_0 \mid b_1 b_2 \cdots b_{52}}$$

**Definition (Normal number).** If  $1 \leq E \leq 2046$ , the bits represent

$$x = (-1)^s (1.b_1 b_2 \cdots b_{52})_2 \times 2^{E-1023}.$$

**Remark.** The leading 1 is not stored (Lecture 10). Sign  $s = 0$  means  $x > 0$ ,  $s = 1$  means  $x < 0$ .

# Special Bit Patterns

| exponent field       | mantissa bits  | value                            |
|----------------------|----------------|----------------------------------|
| $E = 0$              | all $b_j = 0$  | $\pm 0$                          |
| $E = 0$              | some $b_j = 1$ | subnormal: $0 <  x  < 2^{-1022}$ |
| $1 \leq E \leq 2046$ | any            | normal number                    |
| $E = 2047$           | all $b_j = 0$  | $\pm \infty$                     |
| $E = 2047$           | some $b_j = 1$ | NaN (“not a number”, e.g. 0/0)   |

**Remark (Subnormal numbers).** Their value is  $(-1)^s(0.b_1 \cdots b_{52})_2 \times 2^{-1022}$ : they fill the gap between 0 and the smallest normal number, with fewer significant bits.

**Example (Smallest positive normal number  $2^{-1022}$ ).**  $s = 0$ ,  $E = 1$   
( $p = 1 - 1023 = -1022$ ), all  $b_j = 0$ :

$$0 \mid 00 \cdots 01 \mid 00 \cdots 0.$$

# Floating-Point Addition: Example (1/2)

**Example**  $(41.7 \oplus 10.425)$ . From Lecture 10 (rounded up in the last bit):

$$x = \text{fl}(41.7) = +1.010011 \underbrace{0110 \cdots 0110}_{44} 10 \times 2^5.$$

Since  $10.425 = 41.7/4$ , the number  $y = \text{fl}(10.425)$  has the same mantissa and exponent  $5 - 2 = 3$ :

$$y = \text{fl}(10.425) = +1.010011 0110 \cdots 0110 10 \times 2^3.$$

To add, write  $y$  with exponent 5: shift its bits two places to the right.

## Floating-Point Addition: Example (2/2)

The bar marks the end of the 52 stored mantissa bits.

$$x = +1.010011\ 0110 \cdots 0110\ 10 \mid \times 2^5$$

$$y = +0.01\ 010011\ 0110 \cdots 0110 \mid 10 \times 2^5$$

$$x + y = +1.1010000100 \cdots 0 \mid 10 \times 2^5$$

$$x \oplus y = \text{fl}(x + y) = +1.1010000100 \cdots 0 \mid \times 2^5 = 52.125$$

The bits after the bar are 10: an exact tie. Ties go to the neighbor whose last bit is 0 (Lecture 10), here rounding down. So  $x \oplus y \neq x + y$ .

(Check:  $52.125 = 110100.001_2 = 1.10100001_2 \times 2^5$ .)

# Floating-Point Operations

**Definition (Floating-point operations).** For floating-point numbers  $x, y$  the computer returns the exact result, rounded to the nearest double (IEEE standard):

$$x \oplus y := \text{fl}(x + y), \quad x \ominus y := \text{fl}(x - y), \quad x \otimes y := \text{fl}(x \times y), \quad x \oslash y := \text{fl}(x/y).$$

| exact arithmetic | floating-point arithmetic |
|------------------|---------------------------|
| $x + y$          | $x \oplus y$              |
| $x - y$          | $x \ominus y$             |
| $x \times y$     | $x \otimes y$             |
| $x/y$            | $x \oslash y$             |

# Rounding Error of One Operation

**Proposition.** Let  $x, y$  be floating-point numbers,  $*$   $\in \{+, -, \times, /\}$  with  $\odot$  its floating-point version, and  $y \neq 0$  if  $*$   $= /$ . Assume  $x * y = 0$  or  $2^{-1022} \leq |x * y| \leq$  largest double (no overflow, no underflow). Then

$$x \odot y = (x * y)(1 + \epsilon) \quad \text{for some } \epsilon \text{ with } |\epsilon| \leq \frac{1}{2}\epsilon_{\text{mach}} = 2^{-53}.$$

*Proof.* By definition  $x \odot y = \text{fl}(x * y)$ . If  $x * y = 0$ , take  $\epsilon = 0$ . Otherwise the bound  $\text{fl}(z) = z(1 + \epsilon)$ ,  $|\epsilon| \leq \frac{1}{2}\epsilon_{\text{mach}}$ , of Lecture 10 applies to  $z = x * y$ . ■

**Remark.**  $\frac{1}{2}\epsilon_{\text{mach}} = 2^{-53} = 1.11 \dots \times 10^{-16}$ . One operation has a relative error of about  $10^{-16}$  at most. Large errors come from many operations, or from steps that enlarge earlier errors.

# Tracking Errors Through Several Operations

**Remark** (*One factor  $(1 + \epsilon_i)$  per operation*). Example:  $a \otimes b$ , then  $\oplus c$ . By the proposition, with  $|\epsilon_1|, |\epsilon_2| \leq \frac{1}{2}\epsilon_{\text{mach}}$ ,

$$\begin{aligned}(a \otimes b) \oplus c &= (ab(1 + \epsilon_1) + c)(1 + \epsilon_2) \\ &= ab(1 + \epsilon_1)(1 + \epsilon_2) + c(1 + \epsilon_2).\end{aligned}$$

Combine the factors:

$$(1 + \epsilon_1)(1 + \epsilon_2) = 1 + \eta, \quad \eta = \epsilon_1 + \epsilon_2 + \epsilon_1\epsilon_2, \quad |\eta| \leq \epsilon_{\text{mach}} + \frac{1}{4}\epsilon_{\text{mach}}^2.$$

**Remark.** So the computed value is the *exact* value of  $ab + c$  for the changed data  $a(1 + \eta)$ ,  $b$ ,  $c(1 + \epsilon_2)$ . This is the idea of backward error analysis (Lectures 13–14).

# Order of Operations Matters

**Remark (No associative law).** In general  $(x \oplus y) \oplus z \neq x \oplus (y \oplus z)$ .

**Example** ( $x = 1, y = z = 2^{-53}$ ).  $1 + 2^{-53}$  is exactly halfway between 1 and  $1 + 2^{-52}$ ; the tie goes to 1. Hence

$$\begin{aligned}(1 \oplus 2^{-53}) \oplus 2^{-53} &= 1 \oplus 2^{-53} = 1, \\ 1 \oplus (2^{-53} \oplus 2^{-53}) &= 1 \oplus 2^{-52} = 1 + 2^{-52}.\end{aligned}$$

**Remark.** The order of a sum can change the last digits of the result, e.g. when a compiler or a parallel program reorders it.

# Loss of Significant Digits: Example (1/2)

Subtraction of nearly equal numbers:  $x - y$  with  $x \approx y$ .

**Example** (*Relative error of  $x - y$  when  $x \approx y$* ). The bar marks the end of the 52 stored mantissa bits:

$$\begin{aligned}x &= 1 + 2^{-52} + 2^{-53} + 2^{-54} = +1.0 \cdots 01 \mid 110 \cdots \times 2^0 \\y &= 1 + 2^{-54} = +1.0 \cdots 00 \mid 010 \cdots \times 2^0\end{aligned}$$

Exact arithmetic:

$$x - y = 2^{-52} + 2^{-53} = 3 \times 2^{-53}.$$

## Loss of Significant Digits: Example (2/2)

Rounding: for  $x$  the dropped bits 11 are more than half a unit, so round up; for  $y$  the dropped bits 01 are less than half, so round down:

$$\text{fl}(x) = +1.0 \cdots 010 \mid \times 2^0 = 1 + 2^{-51}$$

$$\text{fl}(y) = +1.0 \cdots 000 \mid \times 2^0 = 1$$

$$\text{fl}(x) \ominus \text{fl}(y) = +0.0 \cdots 010 \mid \times 2^0 = 2^{-51}$$

The last difference is exact (no new rounding). Relative error:

$$\left| \frac{2^{-51} - 3 \times 2^{-53}}{3 \times 2^{-53}} \right| = \frac{4 - 3}{3} = \frac{1}{3}.$$

# Where the Error Comes From

**Remark (Storage errors).** Each input is stored with a small error:

$$\text{fl}(x) - x = 2^{-54}, \quad \text{fl}(y) - y = -2^{-54}$$

(relative errors below  $2^{-53}$ ). (Check:

$$2^{-51} - 2^{-52} - 2^{-53} - 2^{-54} = (8 - 4 - 2 - 1) 2^{-54}.)$$

**Remark (Effect of the subtraction).** The error of the difference is  $2^{-54} + 2^{-54} = 2^{-53}$ , while the exact difference is only  $3 \times 2^{-53}$ : ratio 1/3. The subtraction removes the equal leading bits, so the storage errors, which sat in bit 53, now make up the leading bits of the result.

**Remark.** The subtraction step adds no error (it is backward stable, Lecture 13); the error was made when the inputs were rounded. Problem sensitivity versus algorithm error is the topic of Lectures 12–14.

# Looking Ahead

- **Next lecture:** Loss of Significant Digits; Stability